Code Summarization Beyond Function Level

Vladimir Makharev
Innopolis University, AIRI
Innopolis, Russia
v.makharev@innopolis.university

Vladimir Ivanov
Research Center of the Artificial Intelligence Institute
Innopolis University
Innopolis, Russia
v.ivanov@innopolis.ru

2025 IEEE/ACM International Workshop on Large Language Models for Code (LLM4Code) | 979-8-3315-2615-3/25/$31.00 ©2025 IEEE | DOI: 10.1109/LLM4Code66737.2025.00024

***Abstract*—Code summarization is a critical task in natural language processing and software engineering, which aims to generate concise descriptions of source code. Recent advancements have improved the quality of these summaries, enhancing code readability and maintainability. However, the content of a repository or a class has not been considered in function code summarization. This study investigated the effectiveness of code summarization models beyond the function level, exploring the impact of class and repository contexts on the summary quality. The study involved revising benchmarks for evaluating models at class and repository levels, assessing baseline models, and evaluating LLMs with in-context learning to determine the enhancement of summary quality with additional context. The findings revealed that the fine-tuned state-of-the-art *CodeT5+ base* model excelled in code summarization, while incorporating few-shot learning and retrieved code chunks from RAG significantly enhanced the performance of LLMs in this task. Notably, the *Deepseek Coder 1.3B* and *Starcoder2 15B* models demonstrated substantial improvements in metrics such as BLEURT, METEOR, and $BLEU_4$ at both class and repository levels. Repository-level summarization exhibited promising potential but necessitates significant computational resources and gains from the inclusion of structured context. Lastly, we employed the recent SIDE code summarization metric in our evaluation. This study contributes to refining strategies for prompt engineering, few-shot learning, and RAG, addressing gaps in benchmarks for code summarization at various levels. Finally, we publish all study details, code, datasets, and results of evaluation in the GitHub repository available at https://github.com/kilimanj4r0/code-summarization-beyond-function-level.**

***Index Terms*—code summarization, Python, LLM, retrieval-augmented generation, prompt engineering, few-shot learning**

I. Introduction

Code summarization has emerged as a vital area in natural language processing (NLP) and software engineering (SE), aiming to generate concise and meaningful descriptions of source code. The ability to produce human-like summaries of code snippets have been significantly improved by advanced techniques leveraging neural networks, such as sequence-to-sequence models and Transformer-based architectures [1], [2]. These advancements have led to the widespread adoption of automatic code summarization systems, enhancing code readability, maintainability, and overall understanding.

While considerable progress has been made in function-level code summarization, this focus neglects higher levels of abstraction such as classes and repositories. Indeed, summarizing code at the class and repository levels is crucial for comprehending complex and large-scale codebases, as such summarizing encompasses broader context and interactions within the software. Existing models often have limited capabilities in capturing the additional context provided by these higher-level structures, leading to a notable gap in effective techniques for summarizing complex codebases. For instance, when functions involve intricate structures and invoke others, repository-level context becomes essential.

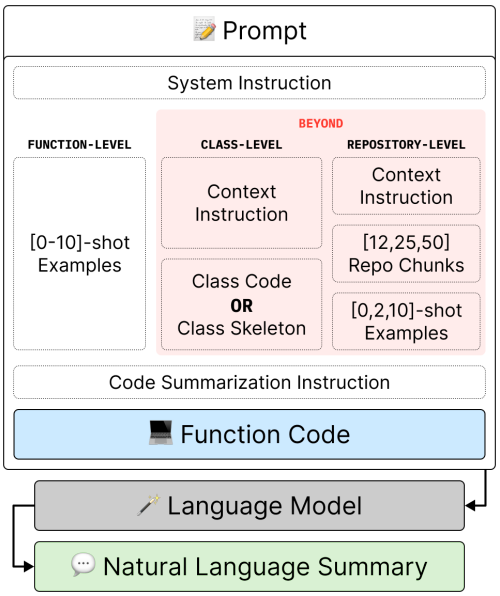

Fig. 1. Code summarization pipeline at function, class, and repository levels, as used in this study.

Addressing this gap is critical for advancing the field and enhancing the utility of code summarization in real-world applications. Incorporating class and repository contexts can potentially provide more comprehensive and meaningful summaries, capturing essential functional and architectural details for developers. Moreover, code summarization models remain insufficiently evaluated beyond individual functions.

This study aims to explore the effectiveness of code summarization models beyond the function level by investigating the impact of additional context from classes and repositories. Fig. 1 presents the overall code summarization pipeline schema used in this work. We hypothesize that this broader context would enrich source code, especially when combined with the utilization of Large Language Models (LLMs) and in-context learning techniques, such as Retrieval-augmented generation (RAG) [3]. Such enrichment will obviously improve the quality of automatically generated code summaries. The additional context is expected to enable models to produce

979-8-3315-2615-3/25/$31.00 ©2025 IEEE
DOI 10.1109/LLM4Code66737.2025.00024

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:19:13 UTC from IEEE Xplore. Restrictio

more comprehensive and concise summaries, enhancing code understanding and maintainability.

To test this hypothesis, we evaluate various code summarization models, including both pre-trained language models and LLMs with in-context learning capabilities, using revised benchmarks that assess performance at function, class, and repository levels. During the evaluation, we used six metrics to analyze the results and determine the broader context effectiveness in code summarization. Our findings revealed improved code summarization quality with the inclusion of additional context and few-shot learning, highlighting the potential of LLMs in this domain.

II. Methodology

This section presents the methodology used to evaluate the effectiveness of code summarization models beyond the function level. Our study addressed a significant gap in the literature by integrating class and repository-level information into code summarization. We aimed to enhance the accuracy and comprehensiveness of automated code summarization techniques. To investigate the research question, **"How effective are code summarization models beyond the function level?"** , we conducted experiments comparing state-of-the-art models at the function, class, and repository levels. The evaluation used widely accepted text generation metrics on benchmark datasets.

For simplicity, we treated methods and functions as equivalent blocks. *Python* was the sole programming language used in this study. A "summary" refers to a concise *English* description of a function code snippet, typically consisting of 1-3 sentences. We chose Python and English due to their widespread use in software development and prevalence in benchmarks, enhancing the relevance and understanding of our research outcomes. Function-level code summarization focused on summarizing individual functions, while class-level and repository-level code summarization provided additional context by summarizing functions within classes and repositories, respectively.

A. Evaluation Datasets

To evaluate code summarization at function, class, and repository levels, we used: the Modified ClassEval dataset and the Modified CodeSearchNet dataset. These datasets are based on the ClassEval benchmark [4] and the CodeSearchNet dataset from the CodeXGLUE benchmark [5], respectively.

Both datasets were obtained using the HuggingFace `datasets` Python package due to its convenient interface. Modifications applied to the datasets are summarized in Table I. As a result, we obtained modified datasets with comparable function summary length distributions depicted in Fig. 2.

1) Modified ClassEval: The Modified ClassEval dataset is derived from the ClassEval benchmark [4], a class-level Python code generation benchmark. This benchmark comprises 100 manually crafted class-level coding tasks, covering 100 classes and 410 methods. We extracted tuples of (context, function code, function summary) for each Python class. The context could be one of the following: (1) empty; (2) class code without the function code to be summarized; (3) class skeleton without the function code to be summarized. Regular expressions were employed to extract summaries from function descriptions and to extract function code to be summarized from class code and skeletons. The distribution in Fig. 2 shows that most function summaries were no longer than 125 characters.

TABLE I
Evaluation Datasets Modification

Dataset	ClassEval	CodeSearchNet
Functions	410	14,918
After modifications: extracting, filtering, and cleaning		
Functions for Evaluation	400	806
Functions for Few-Shot Learning	10	40 (10 per 4 repos)

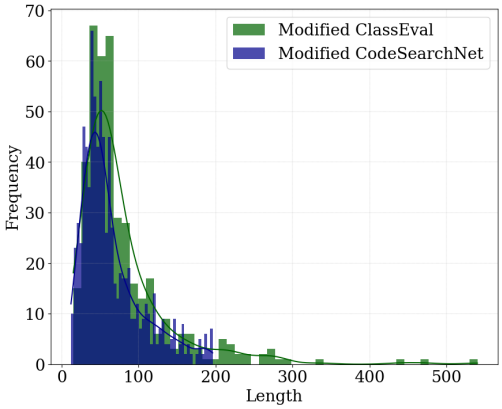

Fig. 2. Function summary length distributions of modified ClassEval and CodeSearchNet datasets.

Next, we randomly selected ten tuples for experiments involving few-shot learning. As a result, we obtained three subsets of the Modified ClassEval dataset, each formatted in JSONL and comprising 400 and 10 tuples of (context, function code, function summary) for evaluation and few-shot learning, respectively.

2) Modified CodeSearchNet: The Modified CodeSearchNet dataset is based on the CodeSearchNet dataset from the CodeXGLUE benchmark [5], which comprises one million filtered pairs of (comment, code) extracted from open-source repositories. We selected the test split comprising 14,918 tuples of (repository name, function code, function documentation) from the Python subset.

Initially, we selected the top repositories from GitHub using the GitHub API, resulting in four repositories after refinement of the selection. Table II presents selected repositories. We employed regular expressions to extract summaries from function documentation and remove documentation and comments from function codes. We then eliminated function summaries shorter than ten or longer than 200 characters, leading to the distribution depicted in Fig. 2.

For each repository, we selected 846 tuples: 806 for evaluation and 40 for few-shot learning (10 per repository). The modified dataset consisted of tuples comprising (repository name, function code, function summary). This modification facilitated incorporating repository context into repository-level code summarization.

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:19:13 UTC from IEEE Xplore. Restrictio

TABLE II
FILTERED REPOSITORIES FOR MODIFIED CODESEARCHNET DATASET (AS OF APRIL 26, 2024)

Repository	Stars	Forks	Functions Extracted	Avg Chunk Size (Chars)	Chunks
apache/airflow	34,500	13,545	455	818	42,925
streamlink/streamlink	9,564	1,074	64	804	3,328
open-mmlab/mmcv	5,601	1,587	50	771	1,940
Azure/azure-sdk-for-python	4,275	2,672	237	839	1,209,693

B. Models

We evaluated five baseline language models and five LLMs on the modified datasets (see Table III). These models were chosen for their distinct architectures, relevance to code summarization tasks, and availability within the Hugging Face model hub.

1) Baseline Language Models: We selected five baseline language models specifically fine-tuned for code summarization tasks. The first two models were based on the CodeTrans [6] architecture: *ct-t5-large-sum* and *ct-t5-large-doc*. The *ct-t5-large-sum* model was fine-tuned to generate concise summaries for Python code snippets, while *ct-t5-large-doc* was optimized for generating detailed documentation from Python code. Both models leveraged the T5-large architecture to capture Python's syntactic and semantic nuances.

The next two models, *codet5-base* and *codet5p-base*, were based on the CodeT5 [7] and CodeT5+ [8] architectures. *Codet5-base* was designed for multi-task code summarization across various programming languages, demonstrating proficiency in generating summaries for diverse code snippets. *Codet5p-base* was specifically trained for Python code summarization tasks, potentially capturing intricate patterns unique to Python code due to its specialized training.

The last model was *pile-t5-large* recently introduced by Sutawika et al. [9]. The authors refined the tokenizers used in standard T5 models by adapting them with the LLaMA tokenizer [13] to improve handling of code-specific syntax, variable names, and structural tokens, ensuring better compatibility with code tokens. The *pile-t5-large* model leverages the T5 architecture fine-tuned on the Pile dataset for code-to-text generation tasks, including code summarization. Trained on diverse code snippets, this model generalizes well across different codebases.

For inference, the baseline models were run using the HuggingFace `transformers` library. We improved the quality and diversity of generated summaries through the `SummarizationPipeline` with a beam search multinomial sampling approach, integrating randomness with structured guidance. A beam search with five beams and multinomial sampling outperformed the greedy approach in initial experiments by generating more diverse and contextually accurate code summaries.

Inference was conducted for each baseline model over both evaluation datasets at the function level. This setup allowed us to assess the performance of models specifically fine-tuned for code summarization tasks.

2) Large Language Models: We selected five LLMs to evaluate their performance in code summarization tasks. The DeepSeek Coder series [10], comprising *deepseek-coder-1.3b*, *deepseek-coder-6.7b*, and *deepseek-coder-33b*, are cutting-edge open-source models tailored for coding across diverse programming languages. These models leverage an extensive pretraining dataset of 2 trillion tokens, predominantly code data, enabling superior performance in code understanding and generation tasks. The different model sizes allow us to evaluate performance across diverse computational resources.

The *starcoder2-15b* model was designed specifically for code-related tasks, utilizing the recent StarCoder2 [11] architecture. *starcoder2-15b* was the first entirely self-aligned code model trained without human annotations or proprietary data. With 15 billion parameters, this model can respond to coding-related instructions in multiple programming languages, making it suitable for code-related tasks.

Lastly, the *llama3-8b* model is based on the LLaMA 3 [12] architecture and supports extended context lengths exceeding 1 million tokens. *llama3-8b* is one of the most recent open-source LLMs, that processes long contexts more effectively compared to other models such as Mistral [14] and Gemma [15]. This feature highlights its potential for enhancing code summarization at the repository level, where long-range context is beneficial.

Notably, the *llama3-8b* model exhibits a substantially larger memory footprint (see Table III), nearly doubling that of its base version with an 8K context length. This doubling is due to its ability to support extended context lengths exceeding 1 million tokens, which is beneficial for repository-level code summarization but requires more computational resources. All models were evaluated using half-precision floating point (`float16`) data types to optimize memory usage without significant loss of precision.

For inference, each LLM was run using the HuggingFace `transformers` library with individual generation configurations. As advised by the authors of the selected models, a greedy decoding approach was uniformly selected for all models. The maximum output length was set at 128 tokens to encompass the majority of summary lengths in the evaluation datasets and to decrease generation time. This configuration promoted consistency and determinism in model output, optimizing efficiency by reducing computational overhead.

Inference of each LLM was conducted at the function, class, and repository levels with varying generation configurations. We used the following context windows to enrich instructions with varying lengths of context: 1M for Llama 3 and 16K for DeepSeek Coder series and StarCoder2. Each LLM was evaluated for function-level code summarization with 1-10 few-shot examples on both evaluation datasets. Subsequently, using the Modified ClassEval dataset, each LLM was assessed for class-level code summarization, with class code or skeleton

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:19:13 UTC from IEEE Xplore. Restrictio

TABLE III
MODELS USED IN THE STUDY

Model	Reference	Shorten Name	Parameters	Memory Usage
Baseline Language Models				
SEBIS/code_trans_t5_large_source_code_sum...	[6]	ct-t5-large-sum	770M	2.75 GB
SEBIS/code_trans_t5_large_code_doc...	[6]	ct-t5-large-doc	770M	2.75 GB
Salesforce/codet5-base-multi-sum	[7]	codet5-base	220M	850 MB
Paul-B98/codet5p_220m_py_sum	[8]	codet5p-base	220M	850 MB
lintang/pile-t5-large-codexglue	[9]	pile-t5-large	783M	2.79 GB
Large Language Models				
deepseek-ai/deepseek-coder-1.3b-instruct	[10]	deepseek-coder-1.3b	1.3B	2.57 GB
deepseek-ai/deepseek-coder-6.7b-instruct	[10]	deepseek-coder-6.7b	6.7B	13.75 GB
deepseek-ai/deepseek-coder-33b-instruct	[10]	deepseek-coder-33b	33B	62.16 GB
bigcode/starcoder2-15b-instruct-v0.1	[11]	starcoder2-15b	15B	29.47 GB
gradientai/Llama-3-8B-Instruct-Gradient-1048k	[12]	llama3-8b	8B	29.98 GB

serving as context. Finally, using the Modified CodeSearchNet dataset, each LLM was integrated into the Naive RAG to evaluate its performance at the repository level.

C. Evaluation

We assessed the quality of the generated code summaries, with the following six widely accepted metrics: BLEU_4 [16], ROUGE_L [17], METEOR [18], BERTScore_{F1} [19], BLEURT [20], and SIDE [21]. These metrics were computed using the HuggingFace `evaluate` library.

- *$BLEU_4$*: Calculated the similarity between the generated and reference summaries based on 1- to 4-gram precision, adjusted by a brevity penalty for shorter sentences.
- *$ROUGE_L$*: Assessed the structural similarity by identifying the longest common subsequence between the generated and reference summaries, considering both precision and recall.
- *METEOR*: Emphasized recall over precision using the harmonic mean of unigram precision and recall, incorporating synonyms and stemming variations.
- *$BERTScore_{F1}$*: Evaluated semantic similarity using embeddings from a pre-trained BERT model, capturing contextual variability in language and combining precision and recall.
- *BLEURT*: A learned evaluation metric that used transfer learning and human annotations to capture semantic and pragmatic aspects of the summaries.
- *SIDE*: A metric tailored for code summarization that modeled the characteristics of suitable and unsuitable code summaries using contrastive learning.

For BERTScore_{F1}, BLEURT, and SIDE the top-performing neural networks recommended by their respective authors: *microsoft/deberta-xlarge-mnli*[1], *BLEURT-20*[2], *side-hard-negatives-fine-tuned*[3], respectively.

In our experiments, the metric results of each model were averaged for each level in each dataset. For the repository level, we also computed the average metrics for each repository project separately. This comprehensive evaluation allows us to assess the performance of different models across various code summarization tasks.

The comparison of code summarization experiments' results was structured as follows:

- Between baselines and LLMs at the function level.
- Between LLMs at the function level with and without 1-10 few-shot examples, at the class level with class code or skeleton, and at the repository level with and without the optimal number of few-shot examples, using varying lengths of context.
- Between LLMs per repository project at the function level and repository level.

This comparison procedure allows for a detailed examination of code summarization performance across various levels, providing insights into the efficacy of different models. Furthermore, Table III introduces shortened names for each chosen model for improved readability.

All experiments were conducted using two NVIDIA A100 GPUs with 80 GB of memory each. The high computational capacity of these GPUs enabled efficient processing of large models and datasets, ensuring that resource constraints did not impede the evaluation.

III. EXPERIMENT DESIGN

We conducted experiments to evaluate code summarization models at the function, class, and repository levels using the methods previously described. Fig. 3 overviews these experiments, most of which involved LLMs. To facilitate reproducibility and contribute to the research community, we published the Modified ClassEval[4] and Modified CodeSearchNet[5] datasets on the HuggingFace Hub.

During the experiments, we employed the inference procedure outlined in Section II-B. Fig. 1 illustrates the code summarization pipeline schema employing a structured prompt with varying levels of context. For the code summarization task, baseline models received only the function code as input, while LLMs were provided with a prompt that included the function code and additional context. The expected output from all models was a concise summary of the function code. To guide the LLMs toward producing relevant summaries, we crafted **a custom system prompt**:

```
You're a specialized AI assisting with
Python code summaries, deeply
knowledgeable in computer science.
```

[1]https://github.com/Tiiiger/bert_score
[2]https://github.com/google-research/bleurt
[3]https://github.com/antonio-mastropaolo/code-summarization-metric
[4]https://hf.co/datasets/sm1rk/modified-classeval-code-summarization
[5]https://hf.co/datasets/sm1rk/modified-codesearchnet-code-summarization

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:19:13 UTC from IEEE Xplore. Restrictio

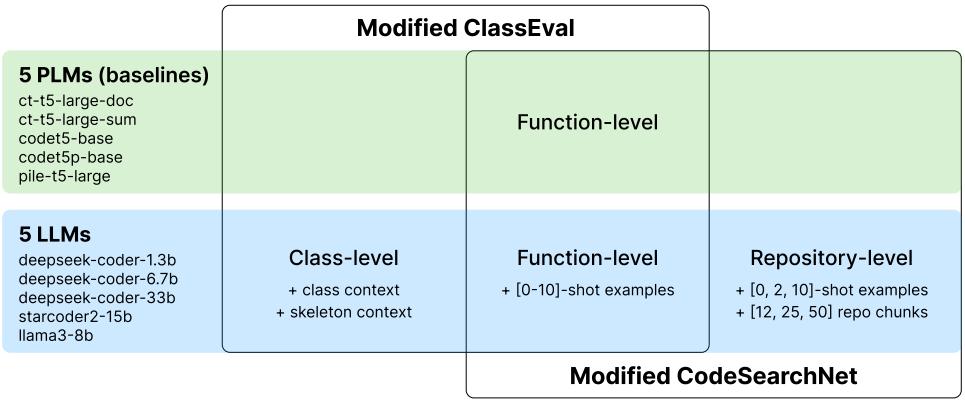

Fig. 3. Experiments conducted in this study.

This prompt was designed to focus the models on generating succinct and accurate code summaries. All prompts in our experiments are provided in detail in the paper's GitHub repository. The most relevant components of these prompts are highlighted below.

A. Function-level Experiments

Function-level code summarization experiments involved both baseline models and LLMs, evaluated using function code as input. Baseline models were assessed solely on this code, while LLMs were tested in few-shot setting, including zero-shot. In the zero-shot setup, LLMs received only the function code and a system prompt without prior examples. In the few-shot setup, 1–10 reference examples were provided as prior message history before the code summarization instruction, guiding the LLM through illustrative cases. **The few-shot examples template** was as follows:

```
{
<few-shot function code>
Concisely summarize the Python code
provided in 1-3 sentences.
<few-shot summary>
} * [1-10]-shot examples
```

The main finding was that LLMs showed consistent improvements as the number of few-shot examples increased. Despite advancements in LLMs, baseline models remained superior in this context.

B. Class-level Experiments

Class-level code summarization experiments focused on evaluating LLMs using either the full class code or a class skeleton[6] as context. We designed **a class context instruction template** to incorporate this additional information:

```
Consider the following class code as
additional context for your response:
<class code or class skeleton>
```

Our findings demonstrated that including the skeleton context consistently improved the performance of the LLMs compared to using the full class code as context.

[6]The class skeleton provides a structured blueprint for the target class, including class-level and function-level information as defined in [4].

C. Repository-level experiments

In our repository-level code summarization experiments, we used LLMs integrated within a Naive RAG pipeline to handle the broader context of entire code repositories. The LLMs were provided with several few-shot examples and code chunks retrieved from the repository as context. Note that some of the code chunks included docstrings though not the target code summary. However, these docstrings did not serve the same purpose as few-shot examples. Few-shot examples were explicitly chosen to demonstrate the structure or style of desired outputs over multiple turns, effectively simulating a conversation or iterative context for the model. By contrast, the docstrings within chunks were simply part of the contextual information provided in a single prompt and did not replicate the guiding role that few-shot examples fulfill.

We implemented a pipeline that included downloading repositories, filtering for Python files, chunking the code into manageable pieces, and constructing a FAISS [22] index to facilitate efficient similarity search. The code chunks were embedded using a high-performing embedding model `BAAI/bge-large-en-v1.5`[7] chosen for its optimal size and high-quality English language support. We selected the top-K most similar chunks based on cosine similarity as context for the LLMs, with K set to 12, 25, and 50. These values were selected to fully utilize the LLM's 16K context window of approximately 50 chunks or 15K tokens, with K halved to facilitate comparison.

A repository context instruction template was designed to present retrieved code chunks with their file paths and content:

```
You have the following repository context,
which includes fragments of code with
their corresponding paths and lines from
the repository:
{
File path: <path of file with code chunk>
File content:
```
<code chunk content>
```
} * [12, 25, 50] repo chunks
```

[7]https://hf.co/BAAI/bge-large-en-v1.5

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:19:13 UTC from IEEE Xplore. Restrictio

This instruction simulated a developer's need to understand a specific Python function within a repository context. The code summarization instruction asked the LLM to summarize the function based on this context, exploring the model's ability to handle large-scale codebases and generate accurate summaries informed by relevant information.

The experiment results indicated that while including code chunks as context did not significantly affect the performance of LLMs when used alone, introducing few-shot examples alongside the context led to noticeable improvements.

IV. Results Analysis and Discussion

The key results are summarized in Table IV, highlighting the effectiveness of code summarization models beyond the function level. We compared the highest-performing baseline model, as determined by metric values, with LLMs in several configurations across selected datasets and levels. Note that running baseline models at the class or repository level yielded meaningless results because these models were trained solely on function code, without natural language instructions as prompts.

The baseline model, *codet5p-base*, demonstrated strong performance due to its state-of-the-art architecture and fine-tuning on the filtered CodeSearchNet dataset [23]. When LLMs were provided with few-shot examples or additional context, they showed significant improvements in generating concise and relevant code summaries.

Including few-shot examples is crucial for enhancing the performance of LLMs in code summarization, as these examples guide LLMs to produce more concise and aligned summaries, reducing their tendency to produce overly explanatory outputs without such guidance. Comparative analysis showed that LLMs with few-shot examples could match or even surpass baseline models in certain metrics, especially when the examples were of high quality. While baseline models retained advantages in certain areas due to fine-tuning, LLMs' adaptability with minimal examples underscored their potential for code summarization beyond the function level. However, the success of LLMs depended on choosing the right examples and providing relevant context.

A. Discussion of specific results

This subsection examines the detailed outcomes across different datasets and levels, emphasizing how the incorporation of contextual information and few-shot learning influences the performance of LLMs compared to the highest-performing baseline model.

In the Modified ClassEval dataset, the baseline model *codet5p-base* achieved a SIDE score of 0.240 at the function level, demonstrating strong alignment with the ground truth summaries. LLMs in zero-shot configurations exhibited higher SIDE scores, indicating less similarity to the ground truth. An example of such LLMs is *deepseek-coder-1.3b*. However, when provided with few-shot examples, LLMs significantly improved their performance. The *starcoder2-15b* model, with 8-shot examples, achieved the highest scores in BERTScore_{F1} (0.757), ROUGE_L (43.62), and BLEU_4 (12.62), outperforming the baseline and other LLMs in these evaluation metrics.

At the class level, the *deepseek-coder-1.3b* model with skeleton context had a higher SIDE score of 0.676, suggesting greater divergence from the ground truth compared to function-level summaries. This unexpected result indicates that simply providing class context without guiding examples may not lead to more concise summaries. The importance of few-shot examples is further emphasized, as models tend to benefit more from structured guidance than from broader context alone. Overall, the Modified ClassEval results demonstrated the efficacy of few-shot learning in improving LLM performance for code summarization tasks. Notably, using only the skeleton structure outperformed entire classes, likely because full class context introduced too much noise, making it harder for the model to extract relevant information for concise summaries.

In the Modified CodeSearchNet dataset, the baseline model *codet5p-base* maintained a competitive SIDE score of 0.075 at the function level. LLMs in zero-shot configurations performed poorly, with *deepseek-coder-1.3b* showing a high SIDE score of 0.659. However, with the inclusion of 10-shot examples, *llama3-8b* and *starcoder2-15b* showed significant improvements. The *llama3-8b* model achieved the lowest SIDE score of 0.041, closely matching the baseline, while *starcoder2-15b* excelled in BERTScore_{F1} (0.738), ROUGE_L (38.15), and BLEU_4 (7.10), indicating superior alignment with the ground truth in these metrics.

At the repository level, LLMs equipped with few-shot examples and additional context outperformed function-level models in certain metrics. The *deepseek-coder-6.7b* model was configured with 10-shot examples and 50 chunks of code context, which is the maximum that filled the LLM's context window. This *deepseek-coder-6.7b* model achieved higher BLEURT (0.568) and BLEU_4 (11.89) scores compared to function-level results. Similarly, the smaller *deepseek-coder-1.3b* model was configured with 10-shot examples and 12 chunks of code context, which outperformed other models in METEOR score (44.08). This result suggests that repository-level code summarization benefits from few-shot examples, leading to improved performance. However, the effectiveness of few-shot examples is critical, as context chunks only did not ensure shorter summaries. The Modified CodeSearchNet results further underscore the value of few-shot learning in guiding LLMs for code summarization beyond the function level.

Unexpectedly, the largest *deepseek-coder-33b* model performed poorly, suggesting a tendency to produce overly detailed summaries, contrary to our goal of concision. Conversely, non-code-specific *llama3-8b* model, performed well in the SIDE metric, indicating strong semantic alignment between the generated summaries and the corresponding code. This performance warrants a reevaluation of the SIDE metric and deeper analysis of the generated summaries, which we plan to explore in future work.

In summary, the combined results from both datasets indi-

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:19:13 UTC from IEEE Xplore. Restrictio

TABLE IV
CODE SUMMARIZATION RESULTS ON MODIFIED CLASSEVAL AND MODIFIED CODESEARCHNET DATASETS

Dataset	Model	Configuration	SIDE↓	BLEURT	BERTScore$_{F1}$	ROUGE$_L$	METEOR	BLEU$_4$
Modified ClassEval								
function-level	codet5p-base	baseline	**.240**	.534	.726	41.43	35.38	8.55
	deepseek-coder-1.3b	LLM + 0-shot	.719	.549	.640	26.04	36.57	3.83
	deepseek-coder-1.3b	LLM + 10-shot	.412	.611	.747	42.80	42.82	11.69
	llama3-8b	LLM + 10-shot	.365	**.620**	.747	42.37	43.20	11.99
	starcoder2-15b	LLM + 8-shot	.269	.616	**.757**	**43.62**	42.40	**12.62**
class-level	deepseek-coder-1.3b	LLM + class context	.693	.576	.684	36.85	46.82	9.02
	deepseek-coder-1.3b	LLM + skeleton context	.676	.586	.691	36.74	**47.87**	10.50
	deepseek-coder-6.7b	LLM + skeleton context	.882	.544	.625	25.85	39.72	5.67
	deepseek-coder-33b	LLM + skeleton context	.934	.536	.605	24.34	38.31	5.10
	llama3-8b	LLM + skeleton context	.834	.560	.629	25.81	41.27	6.46
	starcoder2-15b	LLM + skeleton context	.937	.551	.562	21.33	34.02	4.03
Modified CodeSearchNet								
function-level	codet5p-base	baseline	.075	.502	.721	35.68	33.63	4.30
	deepseek-coder-1.3b	LLM + 0-shot	.659	.481	.587	17.07	28.29	0.79
	deepseek-coder-1.3b	LLM + 2-shot	.142	.529	.706	32.05	35.24	3.26
	deepseek-coder-1.3b	LLM + 10-shot	.068	.539	.724	34.30	34.48	4.46
	llama3-8b	LLM + 10-shot	**.041**	.545	.728	34.60	34.99	5.33
	starcoder2-15b	LLM + 10-shot	.081	.560	**.738**	**38.15**	37.67	7.10
repository-level	deepseek-coder-1.3b	LLM + 2-shot + 12 chunks	.277	.566	.699	35.42	**44.08**	9.70
	deepseek-coder-1.3b	LLM + 0-shot + 12 chunks	.813	.479	.562	20.50	36.40	3.10
	deepseek-coder-6.7b	LLM + 10-shot + 50 chunks	.241	**.568**	.706	36.82	43.65	**11.89**
	deepseek-coder-33b	LLM + 10-shot + 25 chunks	.568	.511	.626	22.60	35.90	3.40
	deepseek-coder-33b	LLM + 10-shot + 50 chunks	.594	.510	.622	22.00	35.40	3.10
	llama3-8b	LLM + 10-shot + 50 chunks	.511	.532	.624	24.80	38.30	6.70
	starcoder2-15b	LLM + 10-shot + 50 chunks	.643	.506	.582	18.50	29.50	2.70

Ground truth value of SIDE for Modified ClassEval is 0.476, for Modified CodeSearchNet is 0.175.
Detailed evaluation results for all experiments are available in the GitHub repository for this study.

cate that few-shot examples significantly enhance the performance of LLMs in code summarization. While baseline models perform well due to specialized fine-tuning, LLMs demonstrate remarkable adaptability when provided with appropriate guidance. The findings highlight the potential of LLMs to effectively summarize code at various levels, provided that such models are equipped with high-quality examples and relevant context.

V. RELATED WORK

The literature on code summarization has predominantly focused on function-level summaries, utilizing encoder-decoder architectures and Transformer-based models to generate concise descriptions of individual functions [24], [7]. These methods have significantly successed in capturing the essence of code snippets but often overlooked the broader context provided by classes and repositories, which is essential for understanding complex codebases. Recent studies have begun to explore code summarization at higher levels, leveraging techniques such as RAG and few-shot learning to incorporate additional context [25], [26]. In addition, [27] explored important research questions regarding file context in code summarization. However, the field still lacks comprehensive evaluation of code summarization models at the class and repository levels. What is more, existing benchmarks fail to assess performance beyond the function level.

LLMs have shown promise in various NLP tasks due to their ability to capture long-range dependencies and contextual information [28], [10]. In code summarization, LLMs with in-context learning capabilities have the potential to generate more informative summaries by leveraging additional context from class and repository structures. Previous work has explored project-specific code summarization using few-shot examples and neural prompt selection [29], but the application of LLMs in this setting remains underexplored. This study builds upon the above advancements by evaluating the effectiveness of LLMs in code summarization beyond the function level, addressing the existing research gap and contributing to the development of models that generate more contextually relevant, concise, and accurate summaries.

VI. LIMITATIONS AND FUTURE WORK

Despite the promising findings, this study has several limitations. First, no established benchmarks exist for code summarization beyond the function level, which hinders direct comparison with the existing approaches and complicates the evaluation. Second, the existing datasets are of subpar quality, which may have affected the training and assessment of the models, potentially limiting their generalizability. Third, Naive RAG pipeline presents limitations in effectively leveraging repository context, which may have impeded performance improvements at the repository level. Naive RAG did not help as much as expected; therefore, we leave the manual investigation of this question for future work. Lastly, prompt engineering is time-consuming, especially when crafting effective few-shot examples. This limitation poses scalability challenges for practical applications. To address the above limitations, we plan to explore promising advanced RAG methods in future work to enchance context utilization and improve code summarization. The examples of such edvanced methods are

ed licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:19:13 UTC from IEEE Xplore. Restriction

RAG utilizing knowledge graphs such as GraphRAG [30] or autonomous LLM agents such as AriGraph [31].

VII. Conclusion

This study demonstrated that code summarization models could effectively operate beyond the function level, particularly when enhanced with additional context and few-shot learning. Fine-tuned language models such as CodeT5+ exceled in code summarization tasks, while incorporating few-shot examples significantly boosted the performance of LLMs such as DeepSeek Coder and Starcoder2. Although repository-level summarization exhibited potential—evidenced by models like DeepSeek Coder—such summarization demanded substantial computational resources and benefited more from structured context than mere repository data. This study lays a foundation for further advancements in the field by providing insights into optimizing code summarization at various levels and highlighting the need for more comprehensive, diverse, and representative benchmarks and datasets. While our study has certain limitations, it represents the first comprehensive exploration of using class- and repository-level context in code summarization, paving the way for future research.

Acknowledgment

All authors were supported by the Research Center of the Artificial Intelligence Institute of Innopolis University.

References

[1] Z. Feng *et al.*, "Codebert: A pre-trained model for programming and natural languages," no. arXiv:2002.08155, Sep. 2020, arXiv:2002.08155 [cs]. [Online]. Available: http://arxiv.org/abs/2002.08155

[2] D. Guo *et al.*, "Unixcoder: Unified cross-modal pre-training for code representation," no. arXiv:2203.03850, Mar. 2022, arXiv:2203.03850 [cs]. [Online]. Available: http://arxiv.org/abs/2203.03850

[3] Y. Gao *et al.*, "Retrieval-augmented generation for large language models: A survey," no. arXiv:2312.10997, Mar. 2024, arXiv:2312.10997 [cs]. [Online]. Available: http://arxiv.org/abs/2312.10997

[4] X. Du *et al.*, "Classeval: A manually-crafted benchmark for evaluating llms on class-level code generation," *arXiv preprint arXiv:2308.01861*, 2023.

[5] S. Lu *et al.*, "Codexglue: A machine learning benchmark dataset for code understanding and generation," no. arXiv:2102.04664, Mar. 2021, arXiv:2102.04664 [cs]. [Online]. Available: http://arxiv.org/abs/2102.04664

[6] A. Elnaggar *et al.*, "Codetrans: Towards cracking the language of silicone's code through self-supervised deep learning and high performance computing," *CoRR*, vol. abs/2104.02443, 2021. [Online]. Available: https://arxiv.org/abs/2104.02443

[7] Y. Wang, W. Wang, S. Joty, and S. C. H. Hoi, "Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," no. arXiv:2109.00859, Sep. 2021, arXiv:2109.00859 [cs]. [Online]. Available: http://arxiv.org/abs/2109.00859

[8] Y. Wang *et al.*, "Codet5+: Open code large language models for code understanding and generation," no. arXiv:2305.07922, May 2023, arXiv:2305.07922 [cs]. [Online]. Available: http://arxiv.org/abs/2305.07922

[9] L. Sutawika, A. Komatsuzaki, and C. Raffel, "Pile-t5," 2024, blog post. [Online]. Available: https://blog.eleuther.ai/pile-t5/

[10] D. Guo *et al.*, "Deepseek-coder: When the large language model meets programming – the rise of code intelligence," no. arXiv:2401.14196, Jan. 2024, arXiv:2401.14196 [cs]. [Online]. Available: http://arxiv.org/abs/2401.14196

[11] A. Lozhkov *et al.*, "Starcoder 2 and the stack v2: The next generation," no. arXiv:2402.19173, Feb. 2024, arXiv:2402.19173 [cs]. [Online]. Available: http://arxiv.org/abs/2402.19173

[12] AI@Meta, "Llama 3 model card," 2024. [Online]. Available: https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md

[13] H. Touvron *et al.*, "Llama: Open and efficient foundation language models," no. arXiv:2302.13971, Feb. 2023, arXiv:2302.13971 [cs]. [Online]. Available: http://arxiv.org/abs/2302.13971

[14] A. Q. Jiang *et al.*, "Mistral 7b," no. arXiv:2310.06825, Oct. 2023, arXiv:2310.06825 [cs]. [Online]. Available: http://arxiv.org/abs/2310.06825

[15] G. Team *et al.*, "Gemma: Open models based on gemini research and technology," no. arXiv:2403.08295, Apr. 2024, arXiv:2403.08295 [cs]. [Online]. Available: http://arxiv.org/abs/2403.08295

[16] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "Bleu: a method for automatic evaluation of machine translation," in *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, P. Isabelle, E. Charniak, and D. Lin, Eds. Philadelphia, Pennsylvania, USA: Association for Computational Linguistics, Jul. 2002, p. 311–318. [Online]. Available: https://aclanthology.org/P02-1040

[17] C.-Y. Lin, "Rouge: A package for automatic evaluation of summaries," in *Text Summarization Branches Out*. Barcelona, Spain: Association for Computational Linguistics, Jul. 2004, p. 74–81. [Online]. Available: https://aclanthology.org/W04-1013

[18] S. Banerjee and A. Lavie, "Meteor: An automatic metric for mt evaluation with improved correlation with human judgments," in *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*, J. Goldstein, A. Lavie, C.-Y. Lin, and C. Voss, Eds. Ann Arbor, Michigan: Association for Computational Linguistics, Jun. 2005, p. 65–72. [Online]. Available: https://aclanthology.org/W05-0909

[19] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "Bertscore: Evaluating text generation with bert," no. arXiv:1904.09675, Feb. 2020, arXiv:1904.09675 [cs]. [Online]. Available: http://arxiv.org/abs/1904.09675

[20] T. Sellam, D. Das, and A. P. Parikh, "Bleurt: Learning robust metrics for text generation," no. arXiv:2004.04696, May 2020, arXiv:2004.04696 [cs]. [Online]. Available: http://arxiv.org/abs/2004.04696

[21] A. Mastropaolo, M. Ciniselli, M. Di Penta, and G. Bavota, "Evaluating code summarization techniques: A new metric and an empirical characterization," no. arXiv:2312.15475, Dec. 2023, arXiv:2312.15475 [cs]. [Online]. Available: http://arxiv.org/abs/2312.15475

[22] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with gpus," no. arXiv:1702.08734, Feb. 2017, arXiv:1702.08734 [cs]. [Online]. Available: http://arxiv.org/abs/1702.08734

[23] H. Husain, H.-H. Wu, T. Gazit, M. Allamanis, and M. Brockschmidt, "Codesearchnet challenge: Evaluating the state of semantic code search," no. arXiv:1909.09436, Jun. 2020, arXiv:1909.09436 [cs, stat]. [Online]. Available: http://arxiv.org/abs/1909.09436

[24] A. Vaswani *et al.*, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[25] R. Li *et al.*, "Starcoder: may the source be with you!" no. arXiv:2305.06161, Dec. 2023, arXiv:2305.06161 [cs]. [Online]. Available: http://arxiv.org/abs/2305.06161

[26] T. Liu, C. Xu, and J. McAuley, "Repobench: Benchmarking repository-level code auto-completion systems," *arXiv preprint arXiv:2306.03091*, 2023.

[27] C.-Y. Su, A. Bansal, and C. McMillan, "Revisiting file context for source code summarization," *Automated Software Engineering*, vol. 31, no. 2, p. 62, 2024.

[28] OpenAI *et al.*, "Gpt-4 technical report," no. arXiv:2303.08774, Mar. 2024, arXiv:2303.08774 [cs]. [Online]. Available: http://arxiv.org/abs/2303.08774

[29] S. Yun, S. Lin, X. Gu, and B. Shen, "Project-specific code summarization with in-context learning," no. 4705650, Jan. 2024. [Online]. Available: https://papers.ssrn.com/abstract=4705650

[30] D. Edge *et al.*, "From local to global: A graph rag approach to query-focused summarization," no. arXiv:2404.16130, Apr. 2024, arXiv:2404.16130 [cs]. [Online]. Available: http://arxiv.org/abs/2404.16130

[31] P. Anokhin *et al.*, "Arigraph: Learning knowledge graph world models with episodic memory for llm agents," no. arXiv:2407.04363, Jul. 2024, arXiv:2407.04363 [cs]. [Online]. Available: http://arxiv.org/abs/2407.04363

d licensed use limited to: MINISTERE DE L'ENSEIGNEMENT SUPERIEUR ET DE LA RECHERCHE SCIENTIFIQUE. Downloaded on March 30,2026 at 17:19:13 UTC from IEEE Xplore. Restrictio